Biconnectivity Analysis using Jen-Schmidt and Slota-Madduri Algorithms

Paarth Jindal
Mehul Agarwal

May 6, 2025

1 Introduction

Efficient analysis of connectivity properties in large-scale undirected graphs is critical in a variety of domains, including network design, infrastructure reliability, and biological systems. In this project, we focus on two fundamental graph problems:

- Determining **biconnectivity** using **Ear Decomposition**.
- Identifying **articulation points** critical to graph disconnection.

We employ two specialized algorithms to tackle these challenges:

- **Jen-Schmidt Algorithm** for constructing an Ear Decomposition.
- **Slota-Madduri Algorithm** for efficient detection of articulation points.

Our evaluation uses sparse graphs in Matrix Market (.mtx) format from the University of Florida Sparse Matrix Collection. The emphasis lies on correctness, performance, and memory efficiency.

2 Problem Statement

Given a sparse undirected graph $G = (V, E)$, our objectives are:

- Verify whether G is **biconnected** using an ear decomposition.
- Identify all **articulation points** using a low-memory, efficient approach.
- Ensure that the solution scales to large graphs with hundreds of thousands of nodes and edges.

3 Slota and Madduri Algorithm for Finding Articulation Points

3.1 Algorithm Overview

The Slota and Madduri algorithm is an efficient approach for identifying articulation points (or cut vertices) in an undirected graph. An articulation point is a vertex whose removal increases the number of connected components in the graph. These points represent vulnerabilities in a network where failure of a single node can disconnect the graph.

Traditional algorithms like Tarjan's use depth-first search (DFS), while the Slota and Madduri algorithm uses a breadth-first search (BFS) approach, which is more amenable to parallelization and can be more efficient for certain graph structures.

3.2 Simple Example

Consider this undirected graph:

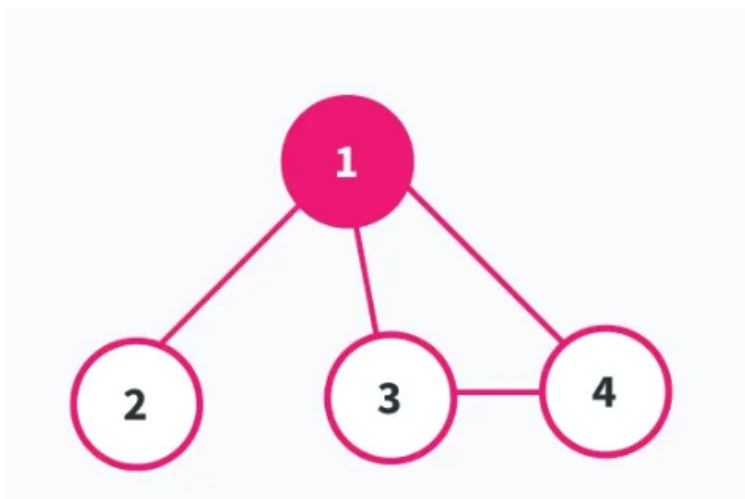

Figure 1: An example image

- Vertex 1 is an articulation point because removing it disconnects vertex 2 from 3 and 4.

3.3 Implementation Details

3.3.1 Key Ideas

- **BFS-Based Approach**: Breadth-First Search (BFS) is used as the foundation for traversal, making the approach suitable for parallelization.
- **Component-wise Processing**: The graph is processed one connected component at a time, allowing independent identification of articulation points.

- **Special Handling for Root Vertices**: Roots in BFS trees are checked separately since their articulation status depends on the number of children.
- **Exclusion BFS**: For each non-root node, we exclude it and perform a BFS to determine whether the rest of its subtree remains connected, helping verify if the node is an articulation point.
- **Cycle Graph Optimization**: If a component is a simple cycle, it is immediately skipped, as cycles have no articulation points.

3.3.2 Data Structures

- `vector<vector<int>> adj`: Stores the graph as an adjacency list.
- `vector<int> P, L, stamp`: Track parent (`P`), level/depth (`L`), and visitation (`stamp`) for BFS.
- `vector<bool> seen, isArt_slota`: Boolean arrays for BFS visitation tracking and storing articulation point results.
- `queue<int> q`: Used for BFS traversal.

3.3.3 Algorithm Flow

1. Loop over all unvisited nodes and perform BFS to extract each connected component.
2. For each component:
   (a) Check if the component is a simple cycle using `is_cycle_graph()`. If so, skip further processing.
   (b) Perform BFS to build the BFS tree and populate `P` (parent) and `L` (level).
   (c) Use `is_root_articulation_slota()` to determine whether the root is an articulation point, based on the number of children.
   (d) For every non-root vertex v in the BFS tree:
      i. For each child w of v, perform `exclusion_bfs()` starting at w on the graph excluding v.
      ii. If this BFS does not reach any ancestor of v, mark v as an articulation point.
      iii. Otherwise, v is not critical for maintaining connectivity.
3. Store articulation point flags in `isArt_slota`.

3.3.4 Function Highlights

- `is_cycle_graph(vector<vector<int>>& adj, vector<int>& component)`: Checks if a given component forms a cycle.

- `is_root_articulation_slota(int root, const vector<vector<int>>& adj)`: Determines articulation status of root by counting direct children.
- `exclusion_bfs(int exclude, const vector<vector<int>>& adj)`: Runs BFS excluding a node to test if the subtree remains connected to any ancestor.
- `calculateVectorMemory(vector<vector<int>>& v)`: Utility to compute the memory usage of the adjacency list or any 2D vector.

3.4 Complexity Analysis

3.4.1 Time Complexity

- Initial BFS per component: $O(V + E)$
- Root check: Up to $O(V \cdot (V + E))$ in worst case
- Non-root exclusion BFS: $O(V^2 \cdot (V + E))$ worst-case

3.4.2 Space Complexity

- Adjacency list: $O(V + E)$
- Auxiliary arrays: $O(V)$
- Total: $O(V + E)$

3.5 Summary

This implementation of the Slota and Madduri algorithm efficiently identifies articulation points using a BFS-first approach. The process is modular, component-wise, and optimized for performance through early cycle detection and root-specific checks. While the worst-case time complexity is quadratic due to exclusion BFS, practical performance is significantly improved with these optimizations. Space complexity remains linear.

4 Schmidt Ear Decomposition Algorithm

4.1 Algorithm Overview

The Schmidt algorithm computes an ear decomposition of a biconnected graph. An ear decomposition is a partition of the graph's edges into a sequence of paths (ears), where:

- The first ear is a cycle.
- Each subsequent ear is a path with endpoints in earlier ears and internal nodes that are new.

Ear decompositions are useful for analyzing the internal structure of biconnected graphs, detecting connectivity, and constructing canonical representations.

4.2 Simple Example

Consider this undirected graph:

[every node/.style=circle, draw] (0) at (0,0) 0; (1) at (1,1.5) 1; (2) at (2,0) 2; (3) at (1,0) 3; (0) – (1) – (3) – (0); (0) – (2) – (3);

A valid ear decomposition:

- First ear (a cycle): $0 \to 1 \to 3 \to 0$
- Second ear: $0 \to 2 \to 3$

4.3 Implementation Details

4.3.1 Key Ideas

- **DFS-Based Construction**: A DFS tree is built to identify tree and non-tree edges.
- **Non-Tree Edge Triggers**: Each non-tree edge indicates a potential new ear.
- **Ear Tracing via Parent Pointers**: When a non-tree edge is encountered, paths are traced back to the lowest common ancestor.
- **Incremental Decomposition**: Ears are added one at a time, gradually covering the graph.

4.3.2 Data Structures

- `vector<vector<int>> g`: Original adjacency list of the graph.
- `vector<vector<int>> dfs_graph`: Stores the DFS tree and added ears.
- `vector<int> visited, numbering`: Tracks visited status and discovery time of each node.
- `vector<vector<int>> ear_decomposition`: Stores the set of ears constructed in order.

4.3.3 Algorithm Flow (Jen-Schmidt)

The Jen-Schmidt algorithm for ear decomposition proceeds as follows:

1. **Initialize DFS Tree**:
   - Start DFS from an arbitrary node.
   - Maintain parent pointers and discovery times.
2. **Identify Non-Tree Edges**:

- During DFS, any edge connecting to an already visited (non-parent) node is a non-tree edge.
- These edges are not part of the DFS tree and signal a possible ear.

3. **Ear Construction**:
   - For each non-tree edge (u, v), trace paths from both u and v to their lowest common ancestor using parent pointers.
   - The combined path forms a new ear, which is added to the decomposition.
4. **Incremental Addition**:
   - Once an ear is added, it is considered part of the base graph.
   - Continue tracing and adding ears until all non-tree edges are processed.
5. **Termination**:
   - The algorithm terminates once all non-tree edges have been processed into ears.
   - The union of the initial DFS tree and all ears gives an ear decomposition of the biconnected graph.

4.3.4 Function Highlights

- `dfs(int u, int parent)`: Builds the DFS tree and records visitation order.
- `traceEar(int u, int v)`: For a non-tree edge (u, v), traces the path from both u and v to their common ancestor to form an ear.
- `findNonTreeEdges()`: Scans the graph for non-tree edges and invokes `traceEar()` to build ears.
- `calculateVectorMemory(vector<vector<int>>& v)`: Measures memory usage of adjacency structures or ear lists.

4.4 Complexity Analysis

4.4.1 Time Complexity

- DFS Traversal: $O(V + E)$
- Ear Construction (in worst-case): Up to $O(E \cdot V)$

4.4.2 Space Complexity

- Graph structures and ear list: $O(V + E)$
- Visitation and numbering arrays: $O(V)$

4.5 Summary

This implementation of the Jen-Schmidt ear decomposition algorithm constructs an ear-based representation of a biconnected graph. It starts with a DFS to build the tree and uses non-tree edges to incrementally form ears. Each ear is derived by tracing the path to a common ancestor, allowing the algorithm to build a complete decomposition. While the worst-case time complexity is $O(E \cdot V)$ due to backtracking, the space complexity remains linear, and the method is practical for medium to large graphs.

5 Experimental Methodology

5.1 Dataset and Environment

- **Datasets**: We used large-scale sparse graphs from the University of Florida Sparse Matrix Collection. The graphs tested ranged in size from tens of thousands up to 10 million edges.
- **System Configuration**: All experiments were conducted on a Linux system with 16GB RAM using C++17 for implementation.

5.2 Preprocessing and Input Parsing

Matrix Market (.mtx) files were parsed using a custom parser that supports:

- Both weighted and unweighted graphs.
- Variable header formats and comment lines.
- Efficient construction of adjacency lists for undirected graphs.

5.3 Memory Profiling

- We used `vector::capacity` and `sizeof` to estimate memory usage.
- All major structures such as adjacency lists, DFS trees, and auxiliary arrays were profiled individually.
- Peak memory usage for each dataset was logged.

5.4 Execution Flow

1. A script was created to iterate over all selected datasets.
2. For each dataset:
    - The number of vertices and edges was recorded.
    - Both the Schmidt (Jen-Schmidt) and Slota-Madduri algorithms were executed.

- Execution time and memory usage were logged.

3. Results were saved in structured CSV format for downstream analysis.

5.5 Result Visualization

- A Python script was used to parse the result logs and generate plots for:
  - Execution time vs. number of edges
  - Memory usage vs. number of edges
- Each plot contains data for both algorithms, enabling side-by-side comparison.

5.6 Figures

Figure 3 and Figure 5 present the time and memory performance of both algorithms across datasets of varying size.

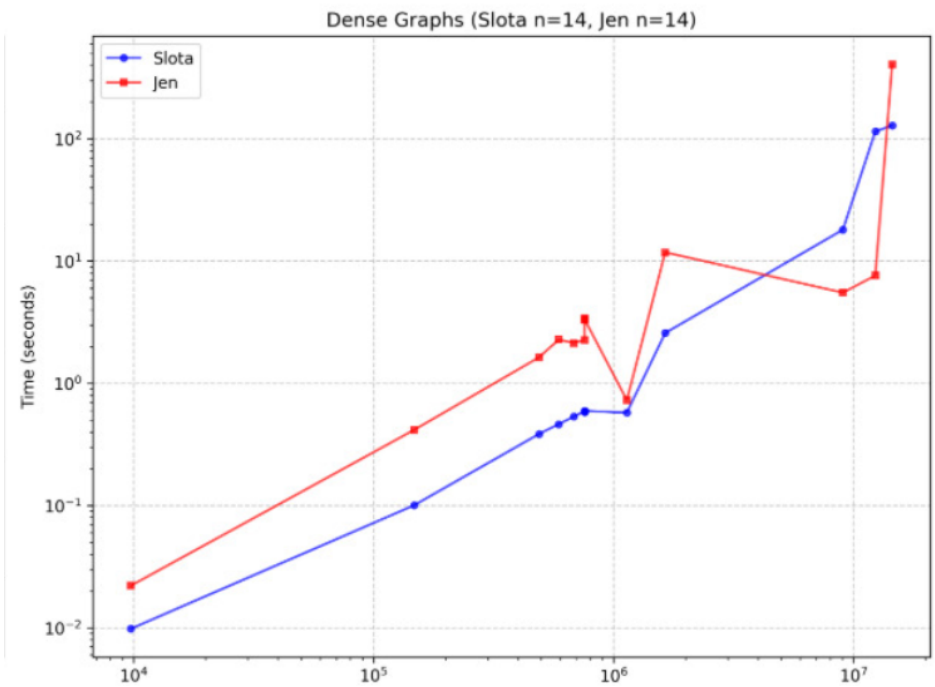

Figure 2: Execution Time vs. Number of Edges for Jen-Schmidt and Slota-Madduri for dense graphs

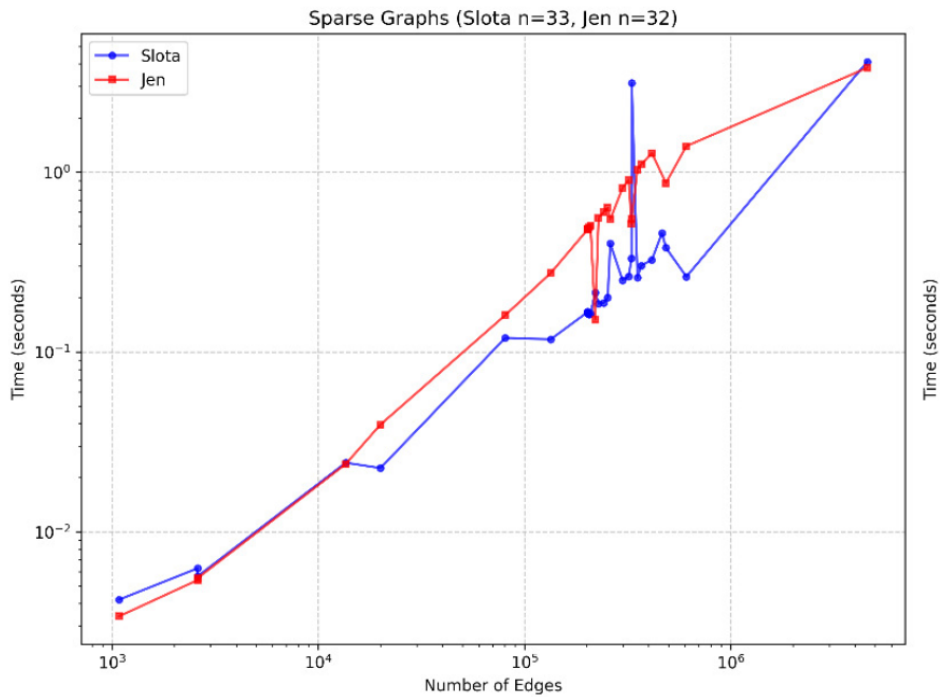

Figure 3: Execution Time vs. Number of Edges for Jen-Schmidt and Slota-Madduri for sparse graphs

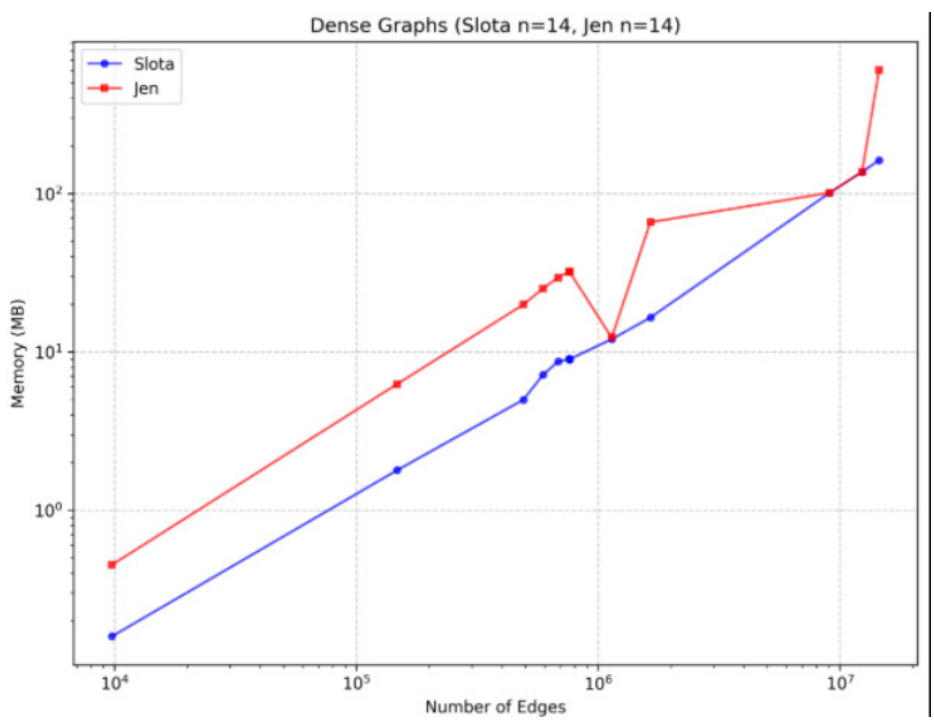

Figure 4: Memory Usage vs. Number of Edges for Jen-Schmidt and Slota-Madduri for dense graphs

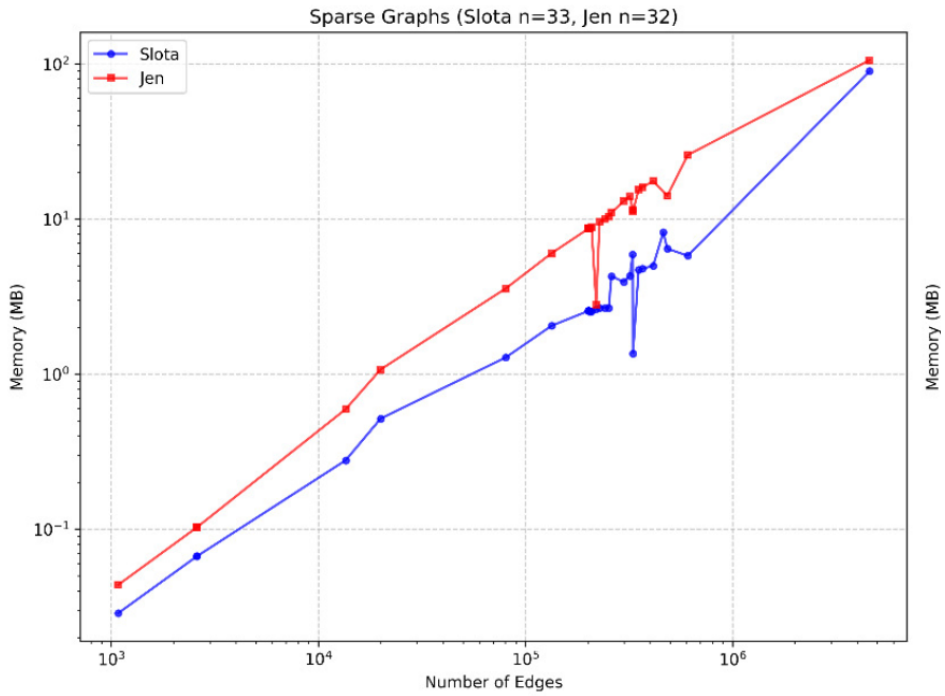

Figure 5: Memory Usage vs. Number of Edges for Jen-Schmidt and Slota-Madduri for sparse graphs

5.7 Comparative Analysis

In this section, we analyze the performance differences between the Slota-Madhuri algorithm and the Jens-Schmidt algorithm, focusing on both time and space complexity. These observations are supported by the plotted results.

Time Complexity. The Slota-Madhuri algorithm generally outperforms the Jens-Schmidt approach in most practical scenarios due to its localized BFS strategy. In moderately structured graphs, the BFS trees are well-branched, allowing efficient reuse of subtrees. This significantly reduces the number of redundant traversals, resulting in better performance. Additionally, since only a limited number of BFS traversals are active at a time, the algorithm scales well with typical input graphs.

In contrast, the Jens-Schmidt algorithm, although asymptotically linear with complexity $O(n + m)$, involves a global DFS traversal and construction of the ear decomposition. This results in higher constant overhead due to the need to maintain and update additional structures like node numbering, visited flags, and paths. In practice, these overheads can make it slower than Slota-Madhuri for many graph types.

However, there are a few specific scenarios where its time complexity degrades significantly.

One such scenario occurs when the input graph has a BFS structure that is very close to a path graph. In this case, the initial BFS tree is minimally branched, resembling a linear

chain. Because Slota-Madhuri requires running a BFS from each child node, this results in nearly $O(n^2)$ BFS calls, leading to a substantial increase in time complexity.

Another scenario where Slota-Madhuri's performance deteriorates is when the graph becomes very dense. As density increases, the number of biconnected components decreases, which also reduces the number of articulation points. Consequently, the algorithm must traverse nearly all child nodes in each BFS call, again increasing the total number of traversals and thus the runtime.

In contrast, the Jens-Schmidt algorithm uses a single DFS traversal followed by ear decomposition, ensuring that each vertex is visited only once. This approach avoids redundant traversals and performs consistently well in sparse graphs due to its $O(n+m)$ behavior, where n is the number of vertices and m the number of edges. Therefore, while Slota-Madhuri often outperforms Jens-Schmidt, these edge-case scenarios highlight where its performance can degrade.

Space Complexity. In terms of space, the Jens-Schmidt algorithm uses more memory due to the need to store several auxiliary structures: the ear decomposition, DFS stack, visited flags, and vertex numbering. These contribute to increased memory overhead, especially for large graphs.

In contrast, Slota-Madhuri is space-efficient—it primarily maintains at most two BFS trees at any point in time, significantly reducing its memory footprint. This lightweight approach gives Slota-Madhuri an advantage in space-constrained environments, especially when compared to the more memory-intensive ear decomposition-based method used in Jens-Schmidt.

6 Conclusion

This project demonstrates the practical implementation of ear decomposition and articulation point detection algorithms for sparse graphs. Jen-Schmidt confirms biconnectivity via structured ears while Slota-Madduri gives efficient articulation detection. Our comparison highlights that Slota-Madduri is more memory-efficient for massive graphs.

7 Future Work

- Parallelize the DFS for improved scalability.
- Apply to dynamic graphs for online biconnectivity testing.